

Driftline.

Persona Journey & Emotion Mapper

A browser-native UX workshop that finds where every user starts to lose patience — and where they recover.

SUBMITTED BY

Pari Gothi

ROLL NUMBER

150096725097

COHORT

Sam Altman

PROBLEM STATEMENT

Where users start to lose patience.

Every digital workflow is a sequence of moments where a user feels something — delighted when one-click checkout works, frustrated when a card form rejects their CVV for the third time, relieved when an order confirmation finally lands. These emotional shifts are the real product, but they are usually invisible. Teams ship features, read complaints later, then guess at which step caused them.

Mapping the emotional journey of a persona — touchpoint by touchpoint, with sentiment scores attached and friction deltas computed — turns those invisible shifts into something a team can argue about, redesign around, and measure. The existing tools fall into two camps: static design files in Figma or Miro that encode nothing structural, or heavyweight cloud platforms like UserTesting and Hotjar that require uploading session data, signing contracts, and waiting for dashboards to render. Neither fits a designer who wants to sketch a journey in the next ten minutes, on sensitive product data, without a vendor account.

Driftline takes a different architectural stance. Every computation — the cubic-bezier trajectory plotter, the friction-delta analyzer, the funnel retention model, the persistence layer — runs entirely inside the user's browser tab. There is no backend, no upload, no account, and no data ever leaves the device. The product opens, the persona's curve renders, and the critical drops are flagged in crimson within a single frame.

TECH STACK

Deliberately small.

Driftline is built on a small, fully owned stack. The case study asks for the trajectory plotter and friction engine to be implemented from scratch — using a chart library would defeat that purpose — so the dependency footprint is kept intentionally minimal.

Runtime

React 18	Functional components and hooks (useState, useEffect, useMemo, useRef). Used purely as the rendering layer.
Vite	Build tool and dev server. Zero-config, instant HMR, ships a static bundle.
JavaScript	Plain JSX, no TypeScript — keeps the case-study codebase scannable in a viva setting.
SVG	Native browser graphics. The trajectory is a single <path>, nodes are <circle>s, gradients drive sentiment colour. No Canvas, no WebGL, no charting library.

Algorithms (hand-written, no library)

Bezier smoothing	Builds the trajectory d-string from touchpoint coordinates using horizontal-tangent midpoints, producing a continuous cubic curve.
Friction deltas	$\Delta F = \text{sentiment}(t) - \text{sentiment}(t-1)$ for every consecutive pair. Drops of two or more trigger the crimson critical-flag.
Funnel retention	Per-step churn coefficient blends sentiment depth with negative ΔF . Retention compounds multiplicatively down the journey.
Sentiment colour map	Linear-interpolated three-stop gradient (coral → amber → tide) drives both the path stroke and every node fill.

State, storage, hosting

State	useState for the persona tree; useMemo for derived analytics so the matrix and HUD recompute only when steps change.
Persistence	window.localStorage. Personas and steps autosave; the workbench restores them on next visit.
Styling	Vanilla CSS with a small token system (Fraunces display, Inter UI, DejaVu Mono utility) and a single brand palette.
Deployment	Vercel. Static output, no server runtime — matches the privacy claim.

COMPONENT ARCHITECTURE

How the parts fit together.

Two layers, cleanly separated: a pure analytics layer with no React in it, and a React rendering layer that subscribes to its output. A single root component swaps between two surfaces — the landing page and the mapper — and a shared SVG renderer powers the emotional curve in both.

React tree

```
Driftline (root; view = landing | app)
|
+-- Landing
| +-- Nav (logo, pricing, launch CTA)
| +-- Hero (headline, sub, animated demo curve)
| +-- Features (plotter, friction, funnel, persistence)
| +-- Pricing (Sketch / Pro / Studio)
| +-- Footer
|
+-- Mapper
+-- AppNav (back, Export Blueprint, Flush Workbench)
+-- PersonaStrip (chips, inline rename, + new)
+-- Stage card
| +-- CardHead (editable persona name & archetype)
| +-- JourneyCurve <-- shared SVG renderer
| +-- TouchpointBuilder (label, stage, sentiment)
+-- Matrix card (Step . Stage . Feel . ΔF . Keep%)
+-- TelemetryHUD (counts . satisfaction . peak . ms)
```

Analytics layer (pure functions)

- **computeGeometry(steps)** Returns {pts, d} — node coordinates and the cubic-bezier path string.
- **sentimentColor(s)** Three-stop linear interpolation; maps any score in [−3, +3] to an RGB.
- **analyze(steps)** Single call returning {rows, satisfaction, peak, latency} — friction deltas, churn-based retention, critical flags.

Render loop

Every step mutation flows through a single `patch()` helper that clones the active persona, applies the change, and returns a new tree. A `useEffect` mirrors the next tree to `localStorage` in the same frame. The shared `JourneyCurve` renders both the landing demo and the mapper's working canvas — which is why the marketing site and the product never drift apart visually.

FEATURES

What the product actually does.

Emotional trajectory engine

- **Cubic-bezier plotter** Touchpoints become a continuous smooth curve with horizontal-tangent control points — no jagged corners.
- **Gradient by sentiment** Coral at frustration, amber at neutral, tide green at delight — gradient applied per segment so the journey reads at a glance.
- **ΔF friction analyzer** Sentiment drops of two or more flag the segment in crimson, ring the offending node, and accent the matrix row.
- **Funnel retention model** Per-step churn coefficient compounds down the journey; projected Keep% updates as you score.
- **Live telemetry** Touchpoint count, satisfaction index, projected retention, peak friction node, and recompute latency in milliseconds.

Interactive workbench

- **Touchpoint builder** Type a label, pick a stage (Discovery / Consideration / Action / Friction Point), slide $-3..+3$ sentiment, press Enter.
- **Inline persona editing** Persona name and archetype edit in place inside the card header; no modal, no save button.
- **Score in the matrix** Sentiment scores edit directly inside the sentiment matrix — the curve updates in the same frame.
- **Hover for detail** Hovering a node reveals the touchpoint, its ΔF , and its retention contribution in a tooltip.
- **Persona strip** Horizontal chip strip across the top — switch active persona, see the archetype underneath the name. One tap adds a new one.

Data flow & product surface

- **JSON blueprint export** One click serialises the entire persona tree to a downloadable JSON file — version control, handoff, or paper appendix.
- **Workbench autosave** Every mutation mirrors to localStorage on the next render; close the tab and the canvas reloads identically.
- **Three-tier pricing** Sketch (free), Pro (\$9/mo), Studio (\$29/mo) — pricing frames the academic project as a real SaaS prototype.
- **Brand-consistent demo** The same SVG renderer powers the landing hero and the mapper, so the marketing demo is the product.

PROJECT AT A GLANCE

By the numbers.

Four numbers that pin down the scope of the build — what was implemented, what was deliberately left out, and how fast the result runs in the browser.

4	0	2	<1
journey stages — Discovery to Friction	external charts or visualization libraries	product surfaces — Landing and Mapper	millisecond typical recompute latency

SCREENSHOTS

The product, captured live.

Screens captured from the deployed instance at driftline-v85g.vercel.app. The curve visible in the landing capture is the same SVG renderer the mapper uses, computed in-browser from the seed persona.

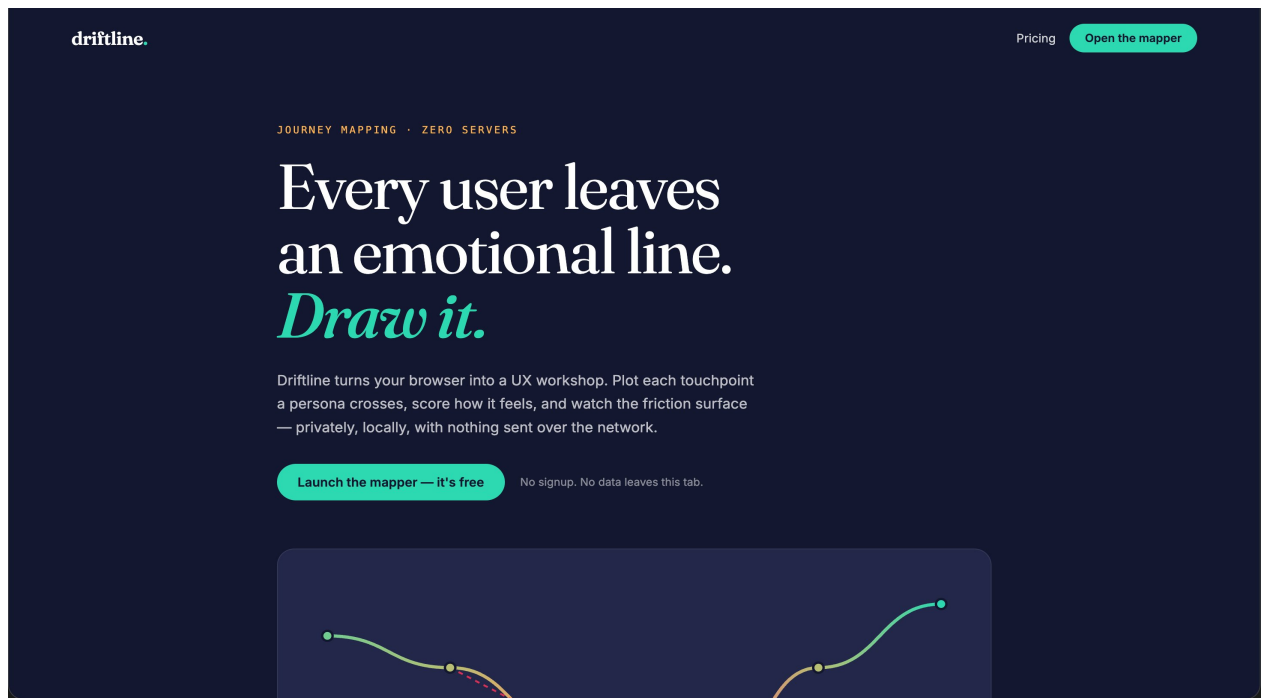


Fig 1 — Landing page hero, with the embedded live emotional-trajectory demo.

SCREENSHOTS

The product, captured live.

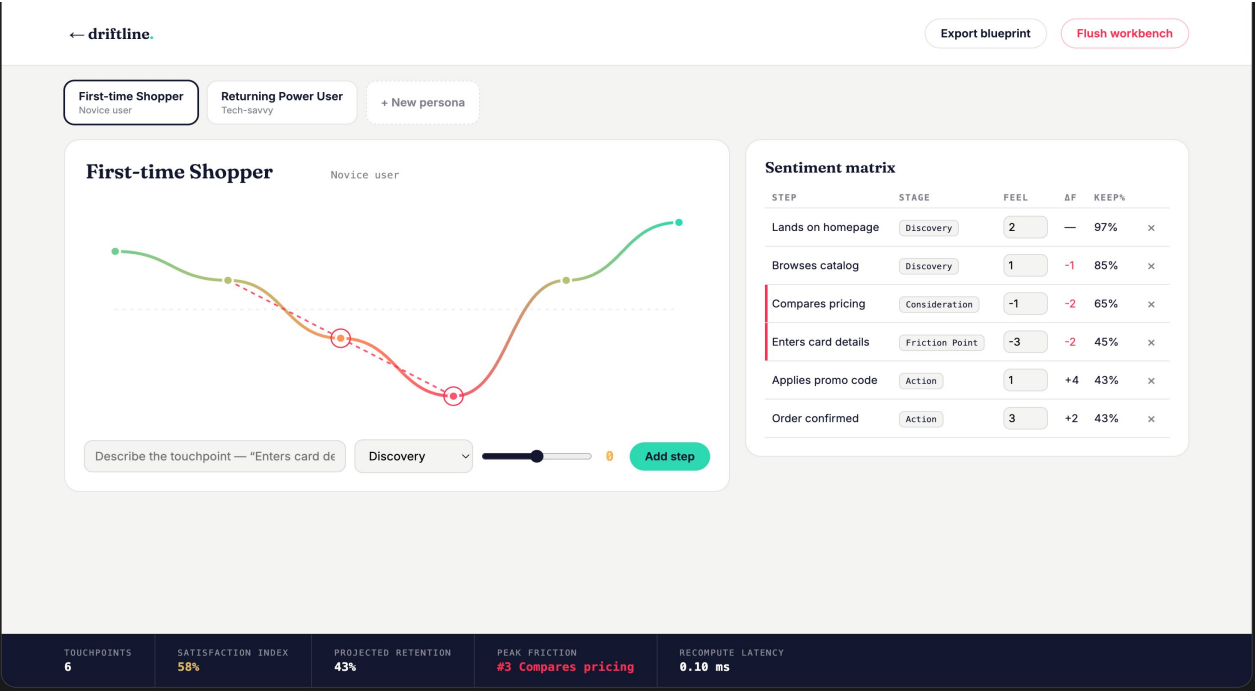


Fig 2 — The mapper: persona strip, journey curve, sentiment matrix, and telemetry HUD.

CONCLUSION

Outcome.

Driftline demonstrates that an emotional-journey workbench can ship as a product rather than as an academic deliverable. Every requirement in the case-study brief — the in-memory behavioral ledger, the fluid emotional trajectory plotter, the interface friction analyzer, the touchpoint builder, the sentiment matrix, and the telemetry HUD — is implemented from scratch in a single React file, with the algorithms (bezier smoothing, ΔF deltas, compounding retention) named in the interface and exposed alongside a live millisecond-precision compute readout.

On the engineering side, the project shows that React hooks, raw SVG, and a small set of pure functions are enough to deliver a fluid, interactive journey-mapping experience without any external charting or animation library. The deliberate choice to avoid d3 or recharts makes the case study honest: every formula that earns marks is visible in the codebase, in plain JavaScript, in fewer than seven hundred lines.

On the product side, the surface — landing, pricing, mapper — frames the academic project as a launchable prototype. The free Sketch tier covers the case-study scope; the paid Pro and Studio tiers sketch a realistic SaaS path for designers and small teams. Whether or not the product ever monetises, the documentation and live deployment turn a semester assignment into something portfolio-shaped.

In one line: Driftline turns any journey into a living curve of feeling — computed where the data already lives.

LINKS

Try it. Read it.

Live demo	driftline-v85g.vercel.app
Source code	github.com/parigothi/driftline
Submitted by	Pari Gothi · Roll no. 150096725097 · Cohort Sam Altman
Institution	ITM Skills University · B.Tech CSE 2025-29 · Semester II